Guia do backend

Como o backend do PokeBox está montado e como as regras de negócio funcionam. Para detalhes de cada campo, use o Swagger em `/docs`. O modelo do banco está em `db.md`.

Ideia do app

O PokeBox é um app de trocas de cartas Pokémon TCG, sem venda nem pagamento. O colecionador cadastra as cartas que tem (coleção), marca as que quer (wishlist), anuncia cartas para troca e faz ofertas nos anúncios dos outros oferecendo cartas da própria coleção. Quando o dono do anúncio aceita, nasce uma troca, e ao concluí-la as cartas mudam de dono.

Os dados das cartas (nome, imagem, variantes, preço) vêm da API pública da TCGdex. O banco guarda só o que é do usuário e uma cópia das cartas usadas.

Como uma requisição anda

```
rota → middleware de login → controller → service → repository → banco (Prisma)
```

Camada	Pasta	Papel
Rotas	`src/routes`	ligam a URL ao controller e dizem se precisa de login
Middlewares	`src/middlewares`	`requireAuth` (login obrigatório), `optionalAuth` (login opcional), `requireRole('ADMIN')`, `errorHandler`
Controllers	`src/controllers`	validam a entrada com zod (`src/schemas`), chamam o service e montam a resposta
Services	`src/services`	regras de negócio e transações
Repositories	`src/repositories`	consultas ao banco (Prisma) e à TCGdex
Regras puras	`src/utils/*Rules.ts`	funções sem banco, fáceis de testar (ex.: "esta oferta cabe no anúncio?")

Quando uma regra é violada, o service lança `new AppError(status, mensagem)`. O `errorHandler` transforma qualquer erro na resposta padrão e traduz os erros do zod (400), do Prisma (duplicado → 409, não encontrado → 404, CHECK do banco → 422) e da TCGdex (502/504).

Toda resposta tem o mesmo formato:

```
{ "success": true, "data": { } }
{ "success": false, "error": "mensagem", "details": [ ] }
```

`details` só aparece em erro de validação.

Login e conta

O login devolve um JWT, válido por 1 dia (`JWT_EXPIRES_IN`), e o front manda esse token em `Authorization: Bearer <token>`. A cada requisição, o `requireAuth` confere o token e busca o usuário no banco. Conta desativada ou suspensa perde o acesso na hora, mesmo com o token ainda válido.

Rota	O que faz
`POST /auth/register`	cria a conta (`username`, `name`, `email`, `password`) e já devolve o token. A senha é gravada com bcrypt
`POST /auth/login`	e-mail e senha → token. Conta suspensa ou inativa → 403
`GET /auth/me`	dados completos do usuário logado (inclui telefone e endereço)
`PATCH /user/me`	edita nome, telefone e endereço
`DELETE /user/me`	desativa a conta. Nada é apagado: o histórico de trocas continua
`GET /auth/admin-check`	só responde 200 para ADMIN; serve para testar o papel

Há dois papéis. `USER` é o padrão; o `ADMIN` só tem poder extra sobre anúncios (bloquear e desbloquear). Ele não mexe na coleção, nas ofertas nem nas trocas dos outros.

Cartas

Rota	O que faz
`GET /cards?name=&page=&limit=&lang=`	busca na TCGdex, paginada. Entende nome de set e ignora palavras que não acham nada (`charizard base x pipi popo` traz os Charizard, com o do Base Set primeiro). Cada item traz `set` e `number`, e a resposta traz `search` explicando como a busca foi entendida
`GET /cards/:id`	detalhe da carta e os anúncios ativos dela. Com login, traz também o bloco `user`: quantas você tem, se está na sua wishlist e se você tem anúncio ativo dela

A busca fica em `utils/cardSearch.ts` (funções puras: `tokenize`, `matchSets`, `rankCards`...). A lista de sets da TCGdex fica 1 dia em memória e o resultado ranqueado de cada busca, 10 min; trocar de página não chama a TCGdex de novo.

Cache: o detalhe de uma carta vai à TCGdex só na primeira vez. Depois fica guardado 24 h no banco (`CardCache`) e 1 h na memória. Se a TCGdex cair, a API usa a última cópia guardada. Sem cópia, responde 502 (fora do ar) ou 504 (demorou mais de 8 s).

Quando uma carta entra na coleção ou na wishlist, os dados dela são gravados uma vez na tabela `Card`, e os itens só apontam para ela.

Coleção

Cada item da coleção é uma carta numa combinação de condição (`NEAR_MINT`, `PLAYED`...), acabamento (`finish`: `NORMAL`, `HOLO`, `REVERSE`) e primeira edição (`firstEdition`).

Rota	O que faz
`GET /collection`	minha coleção. Cada item traz `quantity`, `reservedQuantity` e `available` (o que sobra para negociar)
`POST /collection`	adiciona `cardId`, `quantity`, `finish`, `firstEdition` e, opcionalmente, `condition` e `isPublic`
`PATCH /collection/:id`	muda `quantity`, `condition` ou `isPublic`
`DELETE /collection/:id`	apaga o item
`GET /users/:username/collection`	perfil público de outro colecionador (só itens públicos)

Fluxo do `POST /collection`:

1. Busca a carta na TCGdex (ou no cache). Carta que não existe → 404.
2. Confere se a carta existe nessa variante (`finish` + `firstEdition`). Se não existe → 422.
3. Grava a carta na tabela `Card`, se ainda não estiver lá.
4. Se você já tem a mesma carta com a mesma condição e variante, soma na quantidade. Senão, cria um item novo.

Regras:

- Só o dono vê e mexe nos próprios itens (outro usuário → 403, inclusive o ADMIN).
- A quantidade não pode ficar abaixo do que está reservado em troca nem do que está anunciado (422).
- Item com reserva, já anunciado ou numa oferta aberta ou aceita não pode ser apagado (409). Para tirá-lo do perfil, zere a quantidade ou deixe privado.
- A condição não muda enquanto o item está num anúncio aberto, numa oferta ou numa troca aberta (409). Assim, o que foi anunciado é o que chega.

Wishlist

Rota	O que faz
`GET /wishlist`	minhas cartas desejadas; cada uma com `opportunities`
`POST /wishlist`	adiciona `{ cardId }`
`DELETE /wishlist/:id`	remove (o `id` é o do item da wishlist, não o da carta)

Fluxo do `POST /wishlist`: busca a carta na TCGdex (404 se não existir), grava na tabela `Card` se ainda não estiver lá e cria o item. A mesma carta duas vezes → 409.

A wishlist não tem variante nem condição: quem deseja aceita qualquer versão. O campo `opportunities` conta quantos outros colecionadores têm essa carta pública e disponível, sem contar contas desativadas. É o gancho para o front sugerir "tem gente com essa carta".

Processo de troca

A troca passa por três objetos: anúncio, oferta e troca.

```
ANÚNCIO   ACTIVE                    o dono anuncia uma carta da coleção
   │
   ▼
OFERTA    PROPOSED                  o interessado oferece cartas dele
   ├──► REJECTED                    o dono recusa
   ├──► CANCELED                    o interessado desiste
   ▼
          ACCEPTED                  o dono aceita
   │
   ▼
TROCA     PROPOSED                  o interessado cria a troca: cartas reservadas
   ├──► REJECTED                    o dono recusa: reserva liberada
   ├──► CANCELED                    o interessado cancela: reserva liberada
   ▼
          ACCEPTED                  o dono conclui: as cartas mudam de dono
```

1. Anunciar

`POST /listings` com `collectionItemId`, `quantity` e `description` opcional.

- Só anuncia item da própria coleção (403).
- Somando todos os anúncios abertos (ativos e pausados) do item, a quantidade anunciada não passa do disponível (422).
- Anunciar não reserva nada. A carta continua na coleção e pode ir para outros anúncios dentro do disponível.

`PATCH /listings/:id` muda a quantidade, a descrição ou o status:

Status	Quem	O que acontece com as ofertas
`PAUSED` (pausar)	dono	some do marketplace; ofertas ficam como estão
`ACTIVE` (reativar)	dono	volta ao marketplace
`CLOSED` (fechar)	dono	definitivo. Cancela as ofertas abertas e as aceitas que ainda não viraram troca
`BLOCKED` (bloquear)	só ADMIN	igual a fechar. Desbloquear volta como `PAUSED`, mas as ofertas canceladas não voltam
diminuir a quantidade	dono	cancela as ofertas aceitas que pedem mais do que a nova quantidade

Leitura: `GET /listings` lista os ativos para todo mundo (filtros `cardId` e `username`). `GET /listings/me` lista os meus em qualquer status. `GET /listings/:id` mostra um anúncio pausado, fechado ou bloqueado só para o dono e o ADMIN.

2. Ofertar

`POST /listings/:id/offers`:

```
{
  "quantity": 1,
  "message": "Troco pela minha Mew",
  "items": [{ "collectionItemId": "...", "quantity": 1 }]
}
```

`quantity` é quantas cartas do anúncio você quer. `items` são de 1 a 20 cartas da sua coleção que você dá em troca.

- Só em anúncio ativo de outra pessoa (o próprio → 422; pausado, fechado ou bloqueado → 404).
- `quantity` não passa da quantidade do anúncio.
- As cartas oferecidas são suas (403) e cabem no seu disponível (422).
- Uma oferta aberta por pessoa em cada anúncio (409).
- A oferta não reserva cartas.

3. Responder a oferta

`PATCH /offers/:id/status` com `{ "status": ... }`:

Status	Quem pode
`ACCEPTED`	dono do anúncio
`REJECTED`	dono do anúncio
`CANCELED`	quem fez a oferta

Ao aceitar, o backend confere tudo de novo: o anúncio ainda tem a quantidade pedida e quem ofertou ainda tem as cartas. Se algo mudou → 422.

Uma oferta finalizada não muda mais (422), com uma exceção: a oferta aceita que ainda não virou troca pode ser recusada pelo dono ou cancelada por quem fez, para não ficar presa. Recusar ou cancelar solta as cartas oferecidas.

`GET /offers/me` separa `sent` (as que fiz) e `received` (nos meus anúncios).

4. Criar a troca

Depois que a oferta é aceita, quem fez a oferta chama `POST /trades` com `{ "offerId": "..." }`. A troca nasce como `PROPOSED` e, numa transação:

- reserva as cartas dos dois lados (`reservedQuantity` sobe), e elas não podem ser usadas em nenhuma outra negociação;
- diminui o anúncio pela quantidade da oferta e, se chegar a 0, fecha o anúncio e cancela as ofertas abertas dele;
- cancela as outras ofertas aceitas desse anúncio que não cabem mais;
- reduz os anúncios de quem ofertou que passariam do disponível.

Só existe uma troca por oferta (409).

5. Concluir, recusar ou cancelar a troca

Rota	Quem	Efeito
`POST /trades/:id/accept`	dono do anúncio	as cartas mudam de dono
`POST /trades/:id/reject`	dono do anúncio	a reserva é liberada
`POST /trades/:id/cancel`	quem fez a oferta	a reserva é liberada

O aceite roda numa transação só. Ele trava as cartas, confere que cada uma ainda é de quem entrega e está reservada, tira de quem entrega e soma em quem recebe. Quem recebe ganha a carta no item igual em carta, condição e variante; se não tiver, o backend cria um item público. Se qualquer passo falha, nada muda. Quem entregou continua com o item, mesmo com quantidade 0.

Troca finalizada não muda mais (409). Se duas pessoas agem ao mesmo tempo na mesma troca, uma passa e a outra recebe 409. Recusar ou cancelar a troca não devolve a quantidade que saiu dos anúncios; para tentar de novo, é preciso fazer uma oferta nova.

`GET /trades/me` separa `sent` e `received`. `GET /trades/:id` só é visível para as duas partes (outros, inclusive o ADMIN → 403).

Reserva e disponível

```
available = quantity − reservedQuantity
```

- Só a troca reserva. Anúncio e oferta não reservam, mas respeitam o disponível.
- A reserva entra quando a troca nasce e sai quando ela termina, seja concluída, recusada ou cancelada.
- Operações que podem acontecer ao mesmo tempo no mesmo item (anunciar, ofertar, aceitar) travam a linha do item no banco e rodam uma de cada vez.

Códigos de erro

Código	Quando
400	dado inválido ou JSON quebrado

Código	Quando
401	sem token, token inválido ou vencido, senha errada
403	logado, mas sem permissão (item de outro, rota de admin, conta suspensa)
404	não existe ou você não pode ver
409	conflito: duplicado, item em uso, troca já finalizada
413	corpo acima de 100 KB
422	regra de negócio violada (quantidade, disponível, status errado)
502 / 504	TCGdex fora do ar ou lenta, sem cópia guardada

Para testar

- Swagger: `http://localhost:3000/docs`. Faça login, clique em Authorize e cole o token.
- Usuários do seed: `admin@pokebox.dev`, `ash@pokebox.dev` e `misty@pokebox.dev`, senha `pokebox123`.
- Testes automáticos: `cd backend && npm test`.